

Cartof

- **Limită de timp:** 10 minute
- **Mediu:** un GPU (≈16 GB VRAM), fără internet
- **Dimensiunea soluției:** `solution.ipynb` ≤ 1 MB
- **Spațiu de stocare:** 5 GB

Sarcină

Prietenul dumneavoastră vă propune să jucați un joc de ghicit. El, în calitate de arbitru, alege un cuvânt ascuns dintr-un vocabular fix, iar dumneavoastră trebuie să îl găsiți în cel mult 30 de runde. În fiecare rundă, arbitrul compară două cuvinte și raportează care dintre ele este mai apropiat semantic de cuvântul ascuns. Fiecare joc începe cu perechea fixă `lamp` vs `potato`, deoarece acestea sunt două dintre lucrurile preferate ale prietenului dumneavoastră. Programul dumneavoastră propune apoi un cuvânt nou. Câștigătorul comparației este păstrat și comparat cu următoarea propunere a dumneavoastră. Câștigați jocul în momentul în care propuneți exact cuvântul ascuns. Compararea nu ține cont de diferența dintre literele mari și mici. Fiecare cuvânt pe care îl propuneți trebuie să se afle în `dataset/vocabulary.json`.

Există un exemplu complet în `solution.ipynb`, care include protocolul și încărcarea datelor. Puteți modifica clasa `PublicEmbeddingPlayer`. Programul dumneavoastră este inițializat o singură dată și joacă toate jocurile într-o singură rulare; protocolul creează un nou `PublicEmbeddingPlayer` la începutul fiecărui joc.

Arbitrul

Programul dumneavoastră trimite un obiect JSON Arbitrului, iar Arbitrul răspunde cu un obiect JSON.

Un exemplu complet, în care cuvântul ascuns este afișat doar pentru a explica protocolul:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}          <- matches: game won
```

Rundele sunt indexate de la 1 la 30.

Opțiunile pentru `verdict` sunt `first`, ceea ce înseamnă că `word1` este mai apropiat, `second`, ceea ce înseamnă că `word2` este mai apropiat, sau `same`, ceea ce înseamnă că ambele cuvinte sunt la fel de apropiate de cuvântul ascuns.

`winner_word` este cuvântul păstrat pentru următoarea comparație. În cazul unui `verdict same`, primul cuvânt rămâne.

Dataset

Comun tuturor spliturilor:

- `dataset/vocabulary.json` — 1602 cuvinte unice scrise cu litere mici. Cuvântul ascuns este întotdeauna unul dintre acestea.
- `dataset/public_embeddings.npy` — `float32`, cu forma `(1602, 2560)`. Rândul `i` corespunde cuvântului `i` din vocabular. Acestea sunt embeddinguri *public*; arbitrul utilizează o reprezentare diferită, privată.

Spliturile sunt mulțimi de cuvinte ascunse:

Split	Cuvinte	Răspunsuri	Utilizați-l pentru a
<code>test_public</code>	120	 <code>dataset/test_public.json</code>	rula soluția dumneavoastră și calcula propriul scor
<code>test_leaderboard_a</code>	120	ascunse	clasamentul live
<code>test_leaderboard_b</code>	120	ascunse	clasamentul final

Nu există niciun split `train` — nimic nu este ajustat folosind rânduri etichetate.

Modele furnizate

Două modele de embeddinguri preantrenate sunt incluse în sarcină și pot fi utilizate:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Ambele trebuie încărcate din calea lor locală; un identificator Hugging Face hub precum "BAAI/bge-m3" declanșează o descărcare și eșuează, deoarece evaluarea se desfășoară offline. Fiecare director conține un `example.py` executabil, care prezintă apelul offline.

Biblioteci disponibile: `numpy`, `torch`, `sentence-transformers`. Fără internet, fără descărcări, fără alte pachete.

Ieșire

Niciuna. Aceasta este o sarcină interactivă: soluția dumneavoastră nu scrie niciun fișier de răspuns; aceasta comunică cu arbitrul prin `stdin/stdout`, conform descrierii de mai sus.

Metrică

Un joc în care cuvântul este găsit la runda t primește scorul $1.0 - 0.02 \times \max(0, t - 10)$; un joc care nu este rezolvat în 30 de runde primește scorul 0. Astfel, rundele 1–10 primesc scorul 1.00, runda 20 primește scorul 0.80, iar runda 30 primește scorul 0.60.

Scorul dumneavoastră pentru sarcină este scorul mediu al jocurilor $\times 100$, între 0.00 și 100.00.

Limita de 10 minute reprezintă un singur buget care acoperă pornirea, pregătirea și toate cele 120 de jocuri din setul de testare.

Cum să trimiteți soluția

1. Deschideți `solution.ipynb`, editați `PublicEmbeddingPlayer` și rulați toate celulele pentru a vă asigura că funcționează.
2. Opțional, verificați soluția local: `python local_test.py solution.ipynb --limit 5`. Arbitrul local utilizează embeddingurile *publice*, astfel încât scorul său este doar orientativ.
3. Salvați `solution.ipynb`.
4. Deschideți fila Git din bara laterală din stânga a JupyterLab.
5. Adăugați `solution.ipynb` în staging (pictograma + de lângă acesta).
6. Introduceți un mesaj de commit și faceți clic pe Commit.
7. Faceți clic pe pictograma nor-cu-săgeată-în-sus pentru a efectua push.
8. Reveniți la această pagină a concursului și faceți clic pe Submit, folosind același mesaj de commit pe care l-ați introdus.

Trimiteți exact un fișier, denumit `solution.ipynb`, care să includă toate pregătirile și inferența necesare.